

Comprehensive Defense Study Guide: AI-Powered Eye Tracking

Project Team: Aadarsha, Yudhin, Ishan, Kushal

August 2026

Contents

1	The “Elevator Pitch” & Project Identity	2
2	Deep Dive: The 5-Stage Architecture	2
2.1	Stage 1: Capture & WebRTC	2
2.2	Stage 2: Perception (MediaPipe)	2
2.3	Stage 3: Feature Engineering	2
2.4	Stage 4: The Neural Network (MLP Large v2)	3
2.5	Stage 5: Smoothing & Actuation (PyAutoGUI)	3
3	Mastering the Machine Learning Concepts	4
3.1	Dataset & Data Augmentation	4
3.2	Model Architecture: MLP Large v2	4
3.3	The Optimizer & Loss Function	4
4	Advanced Algorithms	6
4.1	The 9-Point Ridge Regression Calibration	6
4.2	The One Euro Filter (Anti-Jitter)	6
4.3	The “Midas Touch” & Blink Detection (EAR)	6
5	The Defense Grill: Expected Q&A	7

1 The “Elevator Pitch” & Project Identity

What exactly did we build?

We engineered a completely hands-free, AI-driven operating system controller. By looking at a standard consumer webcam, a user can effortlessly control their Windows mouse cursor. They look to move the cursor, and they intentionally blink to click.

Why does this matter? (The Significance)

Historically, eye-tracking required expensive, specialized infrared hardware, isolating individuals who could not afford it. Our project democratizes accessibility for individuals with severe motor disabilities (like Amyotrophic Lateral Sclerosis / ALS or muscular dystrophy) by relying solely on standard, low-cost 720p/1080p webcams already present in consumer laptops.

2 Deep Dive: The 5-Stage Architecture

When a professor asks “Walk me through the pipeline,” this is exactly what happens in the span of 19 milliseconds:

2.1 Stage 1: Capture & WebRTC

During data collection, we needed a fast, reliable way to stream webcam footage to our Python backend. We built a local Flask web server and used **WebRTC** (`navigator.mediaDevices`) in the browser to grab high-speed frames. These frames are converted to Base64 strings and blasted to the backend via **Socket.IO** to avoid HTTP overhead.

2.2 Stage 2: Perception (MediaPipe)

We don’t feed raw images to our AI because images are too “heavy” and slow to process. Instead, we use **Google’s MediaPipe**. MediaPipe is a highly optimized, compiled C++ framework that instantly isolates the face and draws a **478-point 3D mesh** over it.

- **Why 478?:** The base face mesh is 468 points, but MediaPipe adds 10 extra points specifically for refining the iris (pupil) boundaries.

2.3 Stage 3: Feature Engineering

Instead of throwing all 478 points at the AI, we use **NumPy** to calculate specific, meaningful geometry (this is called feature engineering):

- **Head Pitch:** Is the user nodding up and down?

- **Head Yaw:** Is the user shaking their head left and right?
- **Head Roll:** Is the user tilting their head?
- **Normalized Iris Offsets:** Exactly where the pupil is located relative to the corners of the eye, normalized so it works whether the user is sitting close to or far away from the webcam.

2.4 Stage 4: The Neural Network (MLP Large v2)

These lightweight geometric numbers are fed into our custom-trained **Multi-Layer Perceptron (MLP Large v2)**. The MLP is the “brain.” It executes a massive matrix multiplication in just ~ 3 milliseconds (using TensorFlow Lite) and spits out a predicted (X, Y) coordinate on the screen.

2.5 Stage 5: Smoothing & Actuation (PyAutoGUI)

Raw AI predictions shake and jitter. Before we move the mouse, we pass the coordinate through the **One Euro Filter**. Finally, the smoothed coordinate is passed to **PyAutoGUI**, a Python library that physically teleports the Windows OS cursor to that exact pixel.

3 Mastering the Machine Learning Concepts

Professors will heavily scrutinize the AI portion of the project. Memorize these concepts:

3.1 Dataset & Data Augmentation

We manually collected **over 15,000 raw training frames** by looking at moving red dots on our screens. However, 15,000 frames is not enough for deep learning. We mathematically injected **Synthetic Gaussian Noise** (fake static) to the numbers to simulate slight camera jitter and different seating positions. This mathematically tripled our dataset to **45,000 frames**. **Why?** To prevent *overfitting*. If we didn't add noise, the AI would just memorize the exact 15,000 frames instead of actually learning how eyes move.

3.2 Model Architecture: MLP Large v2

Our proposed model is a heavily optimized MLP.

- **He Normal Initialization:** We initialized the starting weights of the network using Kaiming He's method. This prevents the gradients (the learning signals) from vanishing or exploding during early training.
- **Stem & Layers:** It starts wide with 512 neurons, and narrows down through 256, 128, and 64 neuron blocks.
- **Dropout & BatchNormalization:** Dropout randomly turns off neurons during training to force the network to become robust. Batch Normalization standardizes the data between layers so training remains stable.

3.3 The Optimizer & Loss Function

- **Huber Loss:** Normally, people use Mean Squared Error (MSE). We used Huber Loss (with $\delta=0.05$). Huber is a hybrid: it acts like MSE for small errors, but acts like Mean Absolute Error (MAE) for huge errors. This makes our AI highly resilient to "outliers" (e.g., if a frame was blurry and the MediaPipe guess was completely wrong, Huber Loss prevents that single bad frame from ruining our training).
- **Cosine Decay Learning Rate:** Our AI learns using the Adam optimizer. We used a Cosine Decay schedule, which means the AI takes massive learning leaps early on, but smoothly slows down its learning rate (like a cosine curve) as it gets

closer to the perfect answer. This ensures it settles beautifully into the absolute global minimum loss.

4 Advanced Algorithms

4.1 The 9-Point Ridge Regression Calibration

The Problem: Everyone’s monitor is a different size, and everyone sits at a different height. A neural network trained on Aadarsha’s laptop won’t map perfectly to Ishan’s desktop monitor. **The Solution:** We implemented a per-session 9-point calibration. The user looks at 9 dots on the edges and center of their screen. We use Scikit-Learn’s **Ridge Regression** (a polynomial mapping) to warp the AI’s raw predictions to match the specific physical borders of the user’s current monitor. This completely eliminates edge-bias.

4.2 The One Euro Filter (Anti-Jitter)

The Problem: The webcam sensor is noisy. If the mouse cursor perfectly follows the AI prediction frame-by-frame, the mouse will shake violently even if the user is staring perfectly still. **The Solution:** The **One Euro Filter** is an adaptive low-pass filter. At low speeds (fixation), it heavily dampens the input so the cursor stays perfectly still. At high speeds (when the user quickly glances across the screen), it removes the dampening to prevent input lag. It is drastically superior to a simple moving average.

4.3 The “Midas Touch” & Blink Detection (EAR)

The Problem: The “Midas Touch” problem asks: How does the computer know the difference between the user simply *reading* a button, and actually wanting to *click* it? **The Solution:** We use the **Eye Aspect Ratio (EAR)**. We calculate the physical distance between the top/bottom eyelid landmarks versus the left/right eye corner landmarks. When the user intentionally blinks, the EAR value plummets. Our algorithm detects this temporal drop and triggers a PyAutoGUI left-click.

5 The Defense Grill: Expected Q&A

Q: Why did you use MediaPipe instead of just building a Convolutional Neural Network (CNN) to look at the raw camera pixels?

A: While end-to-end CNNs (like the famous iTracker) are highly accurate, they require immense computational power. Processing full-color 1080p pixel arrays frame-by-frame takes too much time, introducing severe latency that makes mouse control feel laggy. By using MediaPipe’s compiled C++ backend to extract just 478 geometric points, we transition from heavy images to lightweight numerical vectors. This hybrid approach allowed us to achieve state-of-the-art accuracy while maintaining a theoretical 50+ FPS inference speed on a standard CPU.

Q: You claim your software can run at 50+ FPS, but your report says the system runs at 25-30 FPS. Explain this contradiction.

A: There is no contradiction; it is the difference between software latency and hardware limitations. Our entire software pipeline (Capture → Infer → Filter → Actuate) completes in approximately 19 milliseconds. Math dictates that $1000\text{ms} / 19\text{ms} = \sim 52$ Frames Per Second. However, standard consumer webcams physically cap their hardware capture rate at 30 FPS. Our software is waiting on the camera, so our sustained end-to-end throughput is artificially bottlenecked at 25-30 FPS by the hardware.

Q: What was the worst performing model you tried, and why did the MLP Large v2 win?

A: We initially tested basic algorithms like Ridge Regression, which yielded a terrible 149 pixel error because it could not understand the non-linear relationship of a 3D head rotating onto a 2D screen. We scaled up to Conv1D networks and an initial MLP Small (v1), which brought the error down to ~ 38 pixels. By widening the network to 512 neurons, adding Batch Normalization, applying Huber Loss, and implementing Cosine Decay, the MLP Large v2 achieved the winning 27.4 pixel error.

Q: How did you measure your final error of 27.4 pixels?

A: We measured Euclidean pixel error on an 80-20 train-test split of our dataset. On a standard 1920x1080 display, a 27 pixel error means the cursor lands well within the bounding box of a standard desktop icon or web browser button, proving it is viable for actual OS navigation.

Q: What is the biggest limitation of your project right now?

A: First, interaction depth: we currently only support left-clicking, so right-clicking or drag-and-drop requires future gesture implementation. Second, environmental fragility: because it relies on standard RGB webcams, the system will struggle in pitch-black

darkness or extreme glare where the camera cannot distinctly see the iris boundaries. Finally, it requires a brief calibration sequence for every new session.

Q: Did you evaluate other anti-jitter filters like the Kalman Filter?

A: Yes, Kalman Filters are the traditional standard. However, tuning a Kalman Filter’s covariance matrices for human gaze is notoriously difficult because human eyes dart unpredictably (saccades). The One Euro Filter is specifically designed for Human-Computer Interaction (HCI); it uses a dynamically calculated cutoff frequency based purely on the instantaneous velocity of the cursor. It was far easier to tune and provided a superior user experience.

Q: What datasets did you use?

A: While we referenced large-scale public datasets like ETH-XGaze in our literature review to understand appearance-based models, our actual neural network was trained entirely on a custom dataset of over 15,000 valid frames that we collected ourselves using our `web_collector.py` application. This ensured the data perfectly matched our specific MediaPipe extraction pipeline.